

S. Gueye
17/11/2024

Chapitre 6

L1

Programmation Dynamique

L2 Informatique - S3

1. Introduction

La programmation dynamique est une technique algorithmique s'appliquant dans la majorité des cas (mais pas que) à la résolution de problèmes d'optimisation.

Elle repose sur les principes généraux suivants :

1. Caractériser la solution optimale en définissant récursivement son calcul en fonction de la solution optimale de sous-problèmes du même type

2. Calculer la valeur optimale
par un processus dit itératif
ascendant, ou par ^{processus} un processus
dit descendant avec mémorisation.
3. Construire une solution optimale
à partir des informations calculées.

Ce chapitre est une introduction à cette technique. Nous venons à travers l'étude d'un problème les notions suivantes :

- Définition d'une récurrence entre le problème que l'on veut résoudre et des sous-problèmes du même type
- Présentation de l'arbre récursif des appels
- Notion de graphe des appels récursifs
- Analyse de la complexité possible d'un algorithme récursif.

- Notion de récursivité avec mémorisation
- Notion d'algorithme itératif ascendant et analyse de sa complexité.

2. Problème de découpe de barres.

Cissu de Cormen et al. - Algorithmique (Dunod)

On considère une barre de longueur n découposable en sous-barres de longueur $i = 1, 2, \dots, n-1$.

A chaque longueur i est associé un prix p_i donné.

Le problème consiste à savoir comment découper la barre de sorte que le revenu total soit maximal.

Exemple : Pour $n=5$, considérons le tableau des prix suivant :

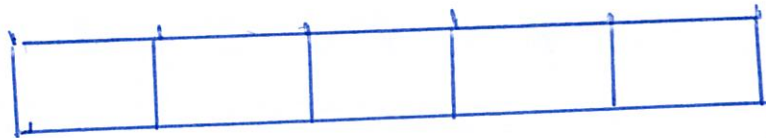
i	1	2	3	4	5
p_i	1	5	8	9	10

14

exemple. Cormen et al. Algorithmes

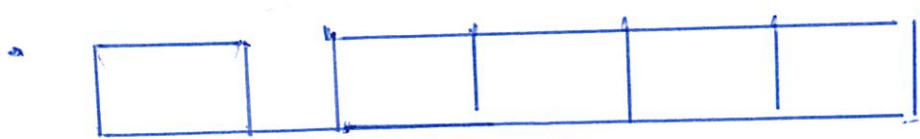
Il y a ~~8~~ façons de découper cette barre. (les autres possibilités, s'y ramènent)

La barre initiale est



La vendre en l'état rapporterait 10.

On pourrait aussi la couper comme suit :



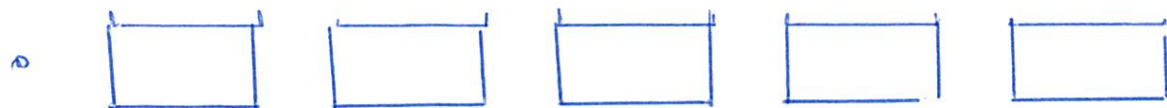
$$\text{revenu} = 1 + 9 = 10$$



$$\text{revenus} = 1 + 1 + 8 = 10$$



$$\text{revenu} = 1 + 1 + 1 + 5 = 8$$



$$\text{revenu} = 1 + 1 + 1 + 1 + 1 = 5$$

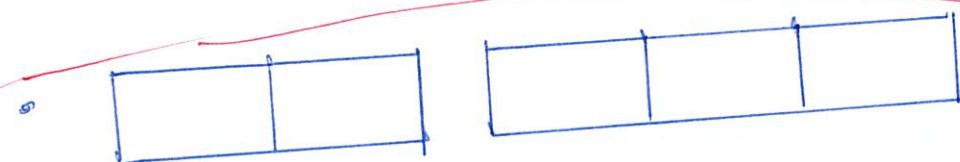


$$\text{revenu} = 1 + 1 + 5 + 1 = 8$$



$$\text{revenu} = 1 + 5 + 5 = 11$$

solution optimale



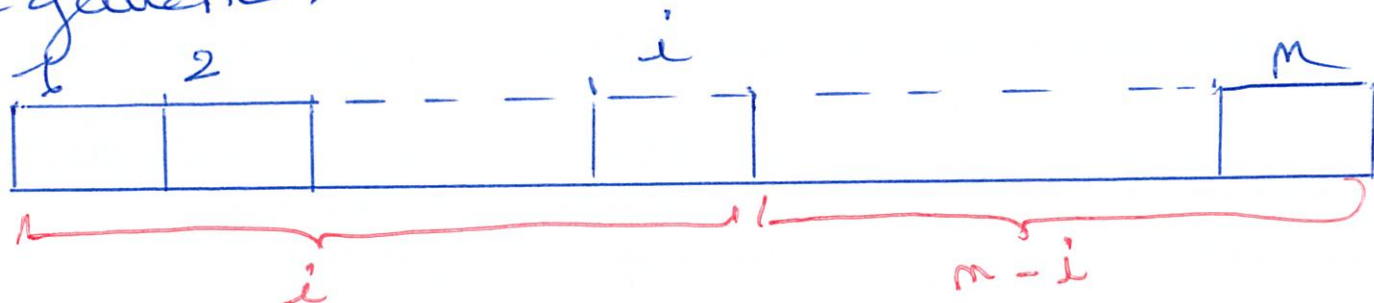
$$\text{revenu} = 5 + 8 = 13$$

□

Notez que le 5^{ème} découpage est équivalent au troisième. Mais cette série de découpage permet de mettre en lumière une récurrence.

On note r_n le revenu maximal que l'on peut obtenir avec une barre de longueur n .

La barre peut être coupée au premier, 1^{er} second, ..., $n-1$ segment. Notons $i=1, 2, \dots, n$ la longueur que l'on coupe à partir de la gauche :



Si la solution optimale est obtenue en coupant une longueur i alors le revenu optimal sera

$$R_n = p_i + R_{n-i}$$

Autrement dit, la longueur restante $(n-i)$ devra nécessairement être découpée de façon optimale. Le découpage optimal de longueur $n-i$ conduira au revenu optimal R_{n-i} qui ajouté au prix du premier tronçon (p_i) donnera R_n .

Comme i n'est pas connu et que l'on maximise alors on a ~~l'on~~ 7

$$\boxed{r_n = \max_{1 \leq i \leq n} \{p_i + r_{n-i}\}}$$

3.1. Calcul de r_n par une méthode récursive.

Une solution simple, mais inadaptée quand n est très grand, serait de calculer $r(n)$ par une méthode récursive, du type suivant

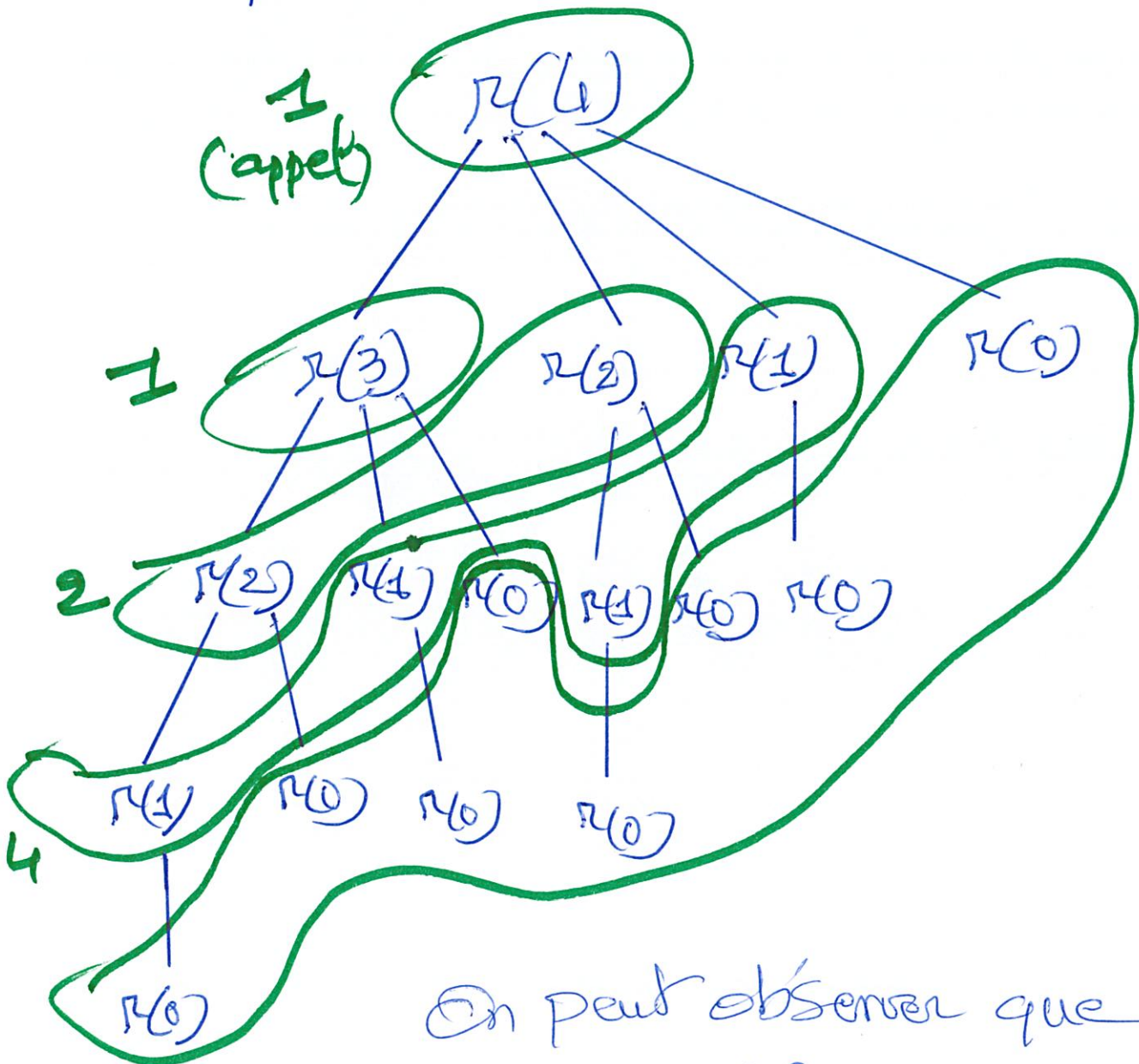
```

r(n)
{
  si (n == 0)
    retourner(0)

  max = -∞
  pour i = 1, 2, ..., n
  {
    val = p_i + r(n-i)
    si (val > max)
      max = val
  }
  retourner(max)
}
```

Analysons la complexité d'une telle 18
procédure.

Dessinons pour $n=4$ l'arbre récursif
des appels à r .



On peut observer que

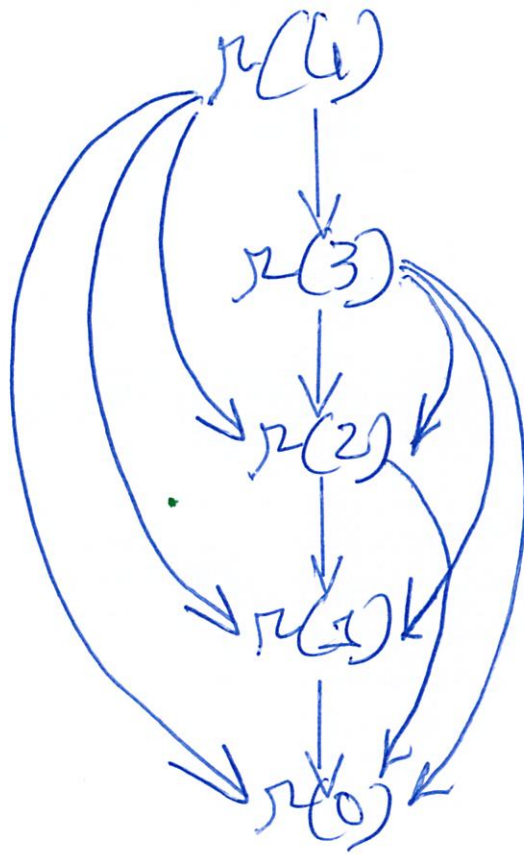
$r(4)$ est appelé 1 fois, $r(3)$ 1 fois,
 $r(2)$ 2 fois, $r(1)$ 4 fois et $r(0)$
8 fois.

La progression semble exponentielle.

19

Now le confirmerons ensuite en le montrant rigoureusement.

On peut également dessiner un graphe représentant ces appels récursifs.



Un arc $a \rightarrow b$ signifie que a appelle b .

Remarquez que le graphe est sans circuit
~~ce qui est en caractéristique~~

Théorème. Soit T_n le nombre ($n \geq 1$) $\llcorner 10$
 d'appels à $r(\bullet)$. On a:
 (ou que l'on s'en rend compte) $n-1$

$$T_n = 1 + \sum_{j=0}^{n-1} T_j$$

preuve: $T_n = \max_{1 \leq i \leq n} \{1 + T_{n-i}\}$

Pour chaque $i = 1, 2, \dots, n$, on a
 un appel récurif à r avec l'argument
 $(n-i)$.

Comme T_n désigne le nombre de fois
 que $r(\cdot)$ est appelé quand on lance $r(n)$
 alors T_{n-i} désignera le nombre de
 fois que r est appelé quand on lance
 $r(n-i)$.

Quand i varie de 1 à n , $(n-i)$
 varie de $n-1$ à 0.

Quand on entre la 1^{ère} fois dans $r(n)$
 cela fait 1 appel récursif.

① où le fait que

$$T_n = I + \sum_{i=1}^n T_{n-i} = 1 + \sum_{j=0}^{n-1} T_j \quad \square$$

Théorème. Soit la suite T_n définie par

$$\begin{cases} T_0 = 1 \\ T_n = 1 + \sum_{j=0}^{n-1} T_j \quad n \geq 1 \end{cases}$$

On a $T(n) = 2^n$

preuve:

$$T_n = \cancel{T_0 + T_1 + \dots + T_{n-1}} + 1$$

$$\sum_{j=0}^{n-2} \cancel{T_j} + \cancel{T_{n-1}} = 2(T_0 + \dots + T_{n-2}) + 1$$

$$2 \sum_{j=0}^{n-3} \cancel{T_j} + 2 \cancel{T_{n-2}} = 2^2(T_0 + \dots + T_{n-3}) + 1 \times 2$$

$$2^2 \sum_{j=0}^{n-4} \cancel{T_j} + 2^2 \cancel{T_{n-3}} = 2^3(T_0 + \dots + T_{n-4}) + 1 \times 2^2$$

$$2^{n-2} \sum_{j=0}^0 \cancel{T_j} + 2^{n-2} \cancel{T_1} = 2^{n-1} T_0 + 1 \times 2^{n-2}$$

En additionnant membre à membre on obtient (12)

$$T_n = 1 + 1 + 2 + 2^2 + \dots + 2^{n-2} + 2^{n-1}$$

$\underbrace{\hspace{15em}}_{2^n - 1}$

$$T_n = 1 + 2^n - 1 = 2^n \quad \square$$

La procédure engendre donc un nombre exponentiel d'appels à RC.

3.2 Méthode récursive avec mémorisation

Elle consiste à stocker les valeurs calculées dans un tableau μ au fur et à mesure de la récursion.

$\mu \text{Memo1}(p, n)$

pour $i = 0, 1, \dots, n$

$\mu[i] = -A$

retourner ($\mu \text{Memo2}(p, n, \mu)$)

$recMemo2(p, m, r)$

13

if $r[m] \geq 0$

return $r[m]$

if $m == 0$

return 0

$max = -\infty$

pour $i = 1, 2, \dots, m$

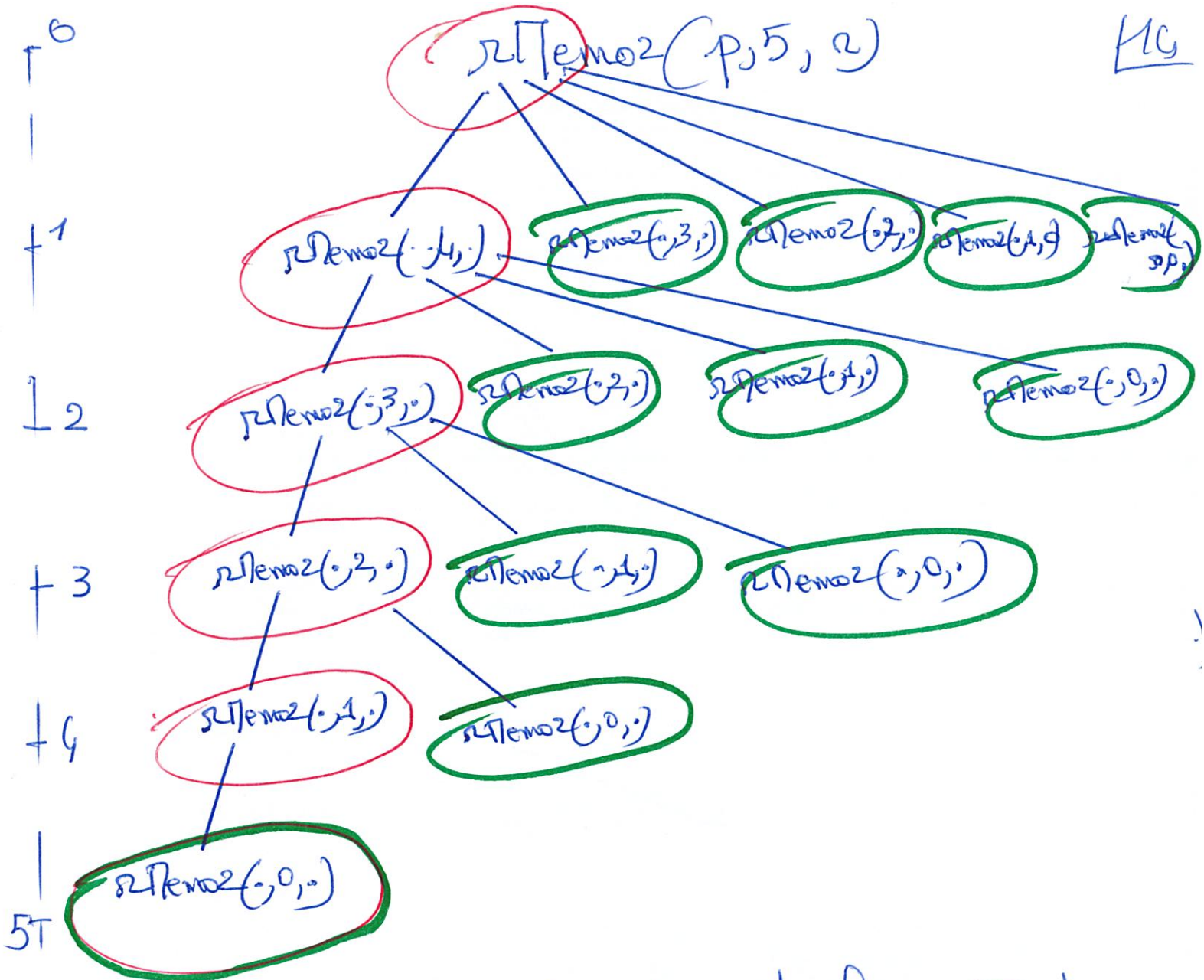
$max = \max(max, p[i] + recMemo2(p, m-i, r))$

$r[m] = max$

return (max)

}

Exemple : pour $m = 5$ $recMemo2$
donnerait l'arbre suivant



En rouge, les nœuds donnant lieu à de nouveaux appels récurifs. En vert ceux ne donnant pas lieu à d'autres appels.

Au niveau 5 on a 1 appel à $nPerm2$

" 4 " 2 appels "

3 " "

1 5 " "

En généralisant pour n quelconque, on observe que le nombre total d'appels est

$$1 + 2 + 3 + \dots + n + 1$$

$$= \frac{n(n+1)}{2} + 1 \in \Theta(n^2).$$

l'appel au niveau 0

Cette complexité peut aussi être ^{obtenue} par une méthode itérative dite ascendante.

3.3. Méthode itérative ascendante

Ascendant(p, n)

ℓ $r[0] = 0$

Pour $j = 1, 2, \dots, n$

ℓ $val = -\infty$

Pour $i = 1, 2, \dots, j$

$val = \max(val, p[i] + r[j-i])$

j $r[j] = val$

j retourner($r[n]$)

2 Ascendant est en $\Theta(n^2)$

15

3.4. Construction de la solution optimale

Les méthodes étudiées jusqu'ici permettent de trouver le revenu optimal, par le découpage optimal.

Dans la ~~formule récurrente~~ suite

$$r_n = \max_{1 \leq i \leq n} \{p_i + r_{n-i}\}$$

notons s_n la taille optimale de la première section

Autrement dit si $i_n = \operatorname{argmax}_{1 \leq i \leq n} \{p_i + r_{n-i}\}$

alors $s_n = i_n$

Si la barre de taille n est coupée à s_n , la section restante est de taille

$$k = n - s_n$$

s_k sera alors la taille optimale de la première section à couper sur cette section sur la sous-barre de longueur k .

Pour $j = 1, 2, \dots, n$ s_j peut être obtenu comme suit :

Ascendant(p, n)

$r[0] = 0$

 pour $j = 1, 2, \dots, n$

$max = -\infty$

 pour $i = 1, 2, \dots, j$

$tmp = p[i] + r[j-i]$

 Si $(tmp > max)$

$max = tmp$

$i_{max} = i$

$r[j] = max, s[j] = i_{max}$

 retourner($r[n]$)

Avec s connu, la solution
peut être affichée comme suit :

Solution (s, m)

⌈ $k = m$

Tant Que ($k > 0$)

⌊ afficher($s[k]$)

⌋ $k = k - s[k]$

⌋